MalNews

Product Requirements Document · Version 2.10 (Draft) · Prepared by Om Parikh, Computer Science @ UC Irvine · November 4, 2026

v2.7: added §3 Threat Model and Security Requirements (SR-1 to SR-9), security tests T6 to T8, and security exit criteria per phase. v2.8: the archive now persists across Pages deploys (FR-21); a keep-alive stops the schedule from being disabled (FR-20); missing CVSS is defined (FR-5); model slugs and feed URLs are marked for verification (FR-1, FR-11); Phase 1 is implemented. v2.9: simplified for Phase 1. SR-1 output checks are deferred, the dry-run mode and fixtures are removed (T2 dropped), and SR-6 is reduced to address checks per hop. v2.10: Phase 2 is implemented. FR-7 adds a CVSS v3.0 fallback and lookup caps; FR-17 uses two-story rows; SR-5 sends the NVD key only to NVD. Items new since v2.6 are marked NEW.

1. Overview

MalNews is a Python pipeline that runs daily on GitHub Actions. It collects the last 24 hours of security news from five sources, adds official severity data, summarizes each story with an AI model through OpenRouter, builds a newspaper-style PDF, and publishes it to GitHub Pages. Requirements: a five-minute read (under 1,150 words), every score and CVE ID from official APIs and not from AI, unattended daily runs including weekends, and $0 cost on the default model. **Design principle** NEW**:** everything fetched from the internet, and everything the AI model returns, is untrusted input. It never gets to run code, fetch files, or reach secrets.

2. Functional Requirements

2.1 Collection

FR-1 Five sources in `sources.yaml` (name, URL, type, tier): CISA advisories (official), BleepingComputer, The Hacker News, Krebs on Security, SecurityWeek (news). Candidate feeds: `cisa.gov/cybersecurity-advisories/all.xml`, `bleepingcomputer.com/feed/`, `feeds.feedburner.com/TheHackersNews`, `krebsonsecurity.com/feed/`, `feeds.feedburner.com/securityweek`. NEW The Feedburner URLs may be retired; confirm every URL on the first live run (T3) and update `sources.yaml`. NVD, CISA KEV, and FIRST EPSS are data APIs, not news sources.

FR-2 Keep items published or updated in the last 24 hours (feedparser). Fetch article text with trafilatura; fall back to the feed excerpt if blocked. Descriptive User-Agent, honor robots.txt, 15 s timeout, 2 retries. All fetching goes through one hardened fetcher (SR-6).

FR-3 A failing source is logged and skipped. Abort only if every source fails.

2.2 Deduplication and Ranking

FR-4 One entry per event. Group by shared CVE ID, then canonical URL, then title similarity (rapidfuzz token set ratio of 85 or higher). Keep every source link.

FR-5 Threat priority = 100 × (0.35 × CVSS/10 + 0.35 × EPSS + 0.30 × KEV); KEV is 1 if listed in CISA's Known Exploited Vulnerabilities catalog, else 0. Weights live in `config.yaml`. NEW A missing CVSS or EPSS value counts as 0 in the formula, and the PDF prints "Score pending" (FR-9), so an unscored CVE never ranks above a scored one by accident.

FR-6 Story score = 0.4 × min(distinct publishers / 5, 1) + 0.4 × (highest linked priority / 100) + 0.2 × impact flag (keywords: outage, ransomware, breach, supply chain, critical infrastructure). Official sources count double. Top 3 = News of the Day.

2.3 Severity and CVE Data

FR-7 Extract CVE IDs from source text with regex `CVE-\d{4}-\d{4,}`, not with AI. For each, query NVD API 2.0 (CVSS v3.1, else v4.0, else v3.0 for older records; NVD's own Primary score first), CISA KEV, and FIRST EPSS. Use a free NVD API key and cache lookups per run. NEW At most 80 NVD lookups per run, most widely reported CVEs first. The PDF shows up to 5 CVEs per story, highest priority first. If NVD fails, the remaining CVEs print "Score unavailable" without further calls.

FR-8 Bands: Critical 9.0–10.0, High 7.0–8.9, Medium 4.0–6.9, Low 0.1–3.9. Print score and band name (not color alone). Link each CVE to `https://www.cve.org/CVERecord?id=<ID>` and `https://nvd.nist.gov/vuln/detail/<ID>`.

FR-9 No NVD score yet: print "Score pending", store the CVE in `state.json`, re-check each run for 14 days. NVD unreachable: print "Score unavailable" and continue.

FR-10 A story enters New Threats (zero-days) when it has a CVE and either an official source or two independent publishers call it a zero-day (keywords: "zero-day", "0-day", "exploited before a patch", "no patch available"). Other KEV-listed items stay in headlines with a Known Exploited badge.

2.4 AI Summaries

FR-11 Model is set by `OPENROUTER_MODEL`; key from `OPENROUTER_API_KEY`. Default `nvidia/nemotron-3.5-lightning:free`. The only other allowed value is `anthropic/claude-haiku-5.5`. Any other value exits at startup. NEW Both slugs, and the FR-13 prices, must be confirmed on openrouter.ai/models before Phase 1 sign-off. The allowlist lives in `config.yaml`, so a renamed model needs no code change.

FR-12 One request per story. The prompt asks for `{"summary": up to 3 sentences, "mitigation": up to 2 sentences or null}` from the first 6,000 characters of article text, using only that text and null when no fix is stated. Do not rely on `response_format` (free model support unconfirmed): parse tolerantly (strip code fences, take the first JSON object with a summary). Retry once on invalid JSON; then use the article's first two sentences, marked as an excerpt.

FR-13 JSON failover (manual): if the free model returns valid JSON on fewer than 95% of requests after the retry during Phase 1 testing, switch `OPENROUTER_MODEL` to `anthropic/claude-haiku-5.5` (standard, not batch; $0.10 in / $0.50 out per 1M tokens, about $0.34 a month at 45 requests a day, about $2 at the cap). This is a config change by the owner. The program does not switch models automatically.

FR-14 On HTTP 429 or a quota error, raise `RateLimitStop`: exit non-zero, publish nothing, keep the previous PDF online, no model fallback. Throttle to the per-minute limit. A cap of 150 requests per run raises `BudgetExceeded`. Expected use is 40 to 60 requests against a 1,000 per day limit. The provider-side credit limit in SR-5 is the hard backstop if this code-level cap fails.

2.5 Story of the Week

FR-15 A week runs Monday 00:00 to Sunday 23:59 PST (UTC-8, fixed). `state.json` holds the current story, its score, the date set, and the last 7 days of story records. Each run scores the last 7 days with FR-6 and replaces the current story when a new one scores at least 20% higher (configurable). The old one moves to Other Headlines and the card shows "Updated <day>". Each Monday it resets to the top story of the last 24 hours.

FR-16 Content from one extended request returning `what_happened`, `who_affected`, `what_changed`, and `mitigation`, plus all source links.

2.6 PDF Layout (Text Only Until Phase 3)

FR-17 US Letter portrait, 3 to 4 pages, under 1,150 words (drop lowest-ranked headlines first). Masthead: MalNews, date, edition number, read time (words / 230), generated time in PST. Order: Story of the Week (full width), News of the Day, New Threats (severity badge, CVE links, Known Exploited badge, "Fix:" line), Other Headlines (2 to 3 sentences), sources. NEW Serif type in two-story rows, laid out as table rows so pages break cleanly between stories. CSS multi-column, flex, and grid blocks jumped whole to the next page in WeasyPrint.

FR-18 Clickable links showing publisher names. Footer: summaries are AI-written from the linked articles; scores come from NVD. Render with WeasyPrint (HTML/CSS columns), fonts bundled in the repo, PDF under 1 MB. No photos until Phase 3. Rendering must meet SR-2.

2.7 Automation and Hosting

FR-19 Repo `security-news-daily`, public (required for free Pages and unlimited Actions minutes). Pages source: "GitHub Actions". URL: `https://<username>.github.io/security-news-daily/`.

FR-20 Workflow `.github/workflows/daily.yml`: cron `0 14 * * *` (6:00 AM PST, 7:00 AM during daylight saving; GitHub may delay a few minutes), every day including weekends, plus `workflow_dispatch` with a `deploy` input (default false). Python 3.12, pip cache, secrets `OPENROUTER_API_KEY` and `NVD_API_KEY`. Job structure, permissions, and action pinning follow SR-3. NEW GitHub disables scheduled workflows in a public repo after 60 days without activity, and commits to the `data` branch may not count. A `keepalive` job runs on the 1st of each month and re-enables `daily.yml` through the API. It is the only job with `actions: write`.

FR-21 Outputs: `site/latest.pdf`, `site/archive/YYYY-MM-DD.pdf`, `site/index.html` (edition list). Deploy only on the schedule or when `deploy` is true. Re-running a day overwrites its file. Keep all editions forever (about 0.3 MB each; review at 800 MB of the 1 GB Pages limit). NEW A Pages deploy replaces the whole site, so the archive cannot live only in `site/`. The `data` branch holds `state.json` and every `archive/*.pdf`. Each run checks it out, adds today's edition, rebuilds `site/` from it, and deploys. After a successful deploy, a separate job commits the new PDF and the validated `state.json` to `data` (SR-3, SR-7).

FR-22	Any failure fails the job (GitHub emails the owner) and publishes nothing. Each run writes a job summary: sources OK/failed, articles, stories after dedupe, requests used, pending CVEs. Runtime under 15 minutes. Logs and summaries are public, so they never contain secrets, headers, or full article text.

3. Threat Model and Security Requirements NEW

Assets. API keys; write access to the repo; integrity of the published edition (people trust its links and scores); the request and dollar budget.

Trust boundaries. Internet → collector · article text → AI model · AI output → renderer · build job → publish job.

Adversaries. (1) Someone who controls article or feed content, through a compromised site, a malicious redirect, or text written to manipulate the model. (2) A compromised Python package or GitHub Action. (3) Anyone who obtains a leaked key.

Goals they would pursue. Steal secrets, push code, publish phishing links under the MalNews name, or run up costs.

The requirements below are ranked by risk: likelihood × impact for this pipeline.

SR-1	**Prompt-injection containment (deferred in v2.9).** Article text goes in the user message only, inside delimiters, and the system prompt says to ignore instructions in it. Replies are parsed but not content-checked. Every link in the PDF still comes from feed metadata, never from AI output. *Accepted risk: a crafted article could get misleading text, including a plain-text URL, into a summary. Revisit before the site is shared widely.*
SR-2	**Safe rendering (no HTML injection, LFI, or SSRF).** Turn on Jinja2 autoescaping and never use `|safe` on external data. Give WeasyPrint a custom `url_fetcher` that serves only `templates/`, `fonts/`, and the image cache. Everything else is denied, including `file://`, `http(s)://`, and `data:`. Links in the PDF must use `https://`; reject `javascript:`, `file:`, and `data:`. Strip PDF metadata down to title and date. *Why: an unescaped `<link rel="attachment" href="file:///proc/self/environ">` in a headline would make WeasyPrint embed the runner's environment, including the API keys, in a public PDF.*
SR-3	**Least-privilege CI/CD.** The workflow sets `permissions: contents: read`. Three jobs, each with only what it needs. **build** holds the API keys, has no write token, and uploads `site/` and `state.json` as artifacts. **publish** has `pages: write` and `id-token: write`, no API keys, and runs in the `github-pages` environment. **data** has `contents: write` and only commits the new edition and a validated `state.json` to the `data` branch. Pin every action to a full 40-character commit SHA. Use `persist-credentials: false`, no `pull_request_target`, `concurrency: malnews` (no overlapping runs), and `timeout-minutes: 20`. Lint workflows with zizmor and actionlint. *Why: the build job processes untrusted input, so it must not also hold a token that can push code.*
SR-4	**Dependency supply chain.** Generate `requirements.txt` with `pip-compile --generate-hashes` and install with `--require-hashes`. CI fails when `pip-audit` finds a known high or critical vulnerability. Dependabot updates pip packages and Actions weekly. Each new dependency is justified in its PR.
SR-5	**Secrets.** Pass keys as `env` only to the step that needs them. NEW The NVD key is sent only to NVD and is dropped if a redirect leads to another host. Never print them, never log request headers, and assume every log is public. Keep `.env` in `.gitignore`. Enable GitHub secret scanning and push protection; CI runs gitleaks on every push. Logs pass through a filter that replaces key values with [REDACTED]. Set a provider-side credit limit on the OpenRouter key ($5 a month) as a hard stop. Rotate keys every 90 days, and immediately after a suspected leak, following a written runbook.
SR-6	**Outbound fetching.** Allow only `http(s)` without credentials, with at most 5 redirects. Before each hop, block hosts that are, or resolve to, private, loopback, link-local (`169.254.0.0/16`), or reserved addresses. Cap responses at 5 MB, always verify TLS (`verify=False` is banned), and ignore proxy settings from the environment. feedparser parses only bytes this fetcher returns. Images (Phase 3): accept JPEG, PNG, or WebP only; set Pillow `MAX_IMAGE_PIXELS`; re-encode to JPEG to drop EXIF data and polyglot payloads.
SR-7	**State and config integrity.** Validate `state.json`, `config.yaml`, and `sources.yaml` against strict schemas (pydantic, unknown fields rejected, size limits) on load. Use `yaml.safe_load` only; no pickle, `eval`, or shell calls built from data. If validation fails, stop the run and publish nothing. Keep state and the archive on the `data` branch so `main` can stay protected.
SR-8	**Public repo hardening.** Branch protection on `main` (PR required, no force-push), `SECURITY.md` with a disclosure contact, Dependabot alerts, CodeQL (Python and Actions), and the OpenSSF Scorecard workflow with its badge in the README.

SR-9	**Publication integrity.** Enforce HTTPS on Pages. Sign each PDF with a build-provenance attestation (`actions/attest-build-provenance`) and list its SHA-256 in `index.html`. `index.html` is static, has no JavaScript, and sets the CSP `default-src 'none'; style-src 'self'; base-uri 'none'; form-action 'none'`. A repo variable `PUBLISH_ENABLED` acts as a kill switch, and a documented takedown step pulls a bad edition within minutes.

4. Testing Without GitHub Pages

- **T1:** pytest offline: dedupe, CVE regex, bands, JSON parsing and retry, Story of the Week replacement, 429 stop.
- **T3:** `--max-requests 10` runs real feeds, APIs, and model (keys in a local `.env`), writes to `site/` locally, deploys nothing.
- **T4:** `python -m http.server -d site 8000` and open `localhost:8000` to check the edition list and links.
- **T5:** run the workflow manually with `deploy=false`; it uploads `site/` as a downloadable artifact with no Pages deploy.
- **T6** NEW**:** attack tests. (a) A headline or summary containing `<img src=file:///etc/passwd>` and `<link rel=attachment>`: the PDF has no attachments, makes no fetches, and shows the text escaped. (b) URLs pointing at `127.0.0.1`, `169.254.169.254`, and a redirect to an internal IP: all are blocked. (c) An oversized response is rejected.
- **T7** NEW**:** CI security gates on every PR: pip-audit, gitleaks, CodeQL, zizmor and actionlint. Any finding at high or above fails the build.
- **T8** NEW**:** a test checks that the build job has no write permissions and that every `uses:` line is pinned to a SHA.

5. Pipeline and Repository

`collect.py` (feeds, article text, CVE IDs) → `enrich.py` (NVD, KEV, EPSS) → `rank.py` (dedupe, scores, News of the Day, Story of the Week) → `summarize.py` → `render.py` → publish. `main.py` runs them in order. The stages live in the `malnews/` package. A shared `safety.py` NEW holds the outbound fetcher (SR-6) and the renderer's URL fetcher (SR-2). Other files: `sources.yaml`, `config.yaml`, `state.json`, `templates/`, `fonts/`, `tests/`, `.github/workflows/daily.yml`, `SECURITY.md`, `.github/dependabot.yml`.

6. Phases and Acceptance Criteria

- **Week 0, Plan (Oct 22 to Oct 25), planning only:** finalize the five sources and verify their feeds, set the config values (weights, thresholds, caps), draft the AI prompts, and write the test plan. Done when these four items are written down and ready to use in Week 1.
- **Phase 1, Collect and summarize (Week 1, Oct 26 to Nov 1):** repo skeleton, `sources.yaml` and `config.yaml` from the plan, collection from the five sources, dedupe, hyperlinks, summaries and fixes, JSON validity check (FR-13), basic text PDF. Done when the offline tests run the whole pipeline end to end with the network and model replaced, a local run outputs a PDF where every story has a summary and working links, and a forced 429 stops cleanly. **Security exit** NEW**:** SR-2, SR-4, SR-5, and SR-6 are in place, and T6 passes. **Status (Oct 30):** implemented. The end-to-end test, the forced 429, and T6 pass in CI. Still open: the live run (T3), which needs the owner's keys, and the FR-13 JSON-validity measurement on the real model.
- **Phase 2, Technical data and newspaper (Week 2, Nov 2 to Nov 8):** CVE extraction, NVD/KEV/EPSS, bands, priority, Score pending, New Threats, News of the Day, Story of the Week, multi-column layout, read time. Done when 10 sample CVEs match NVD exactly, a 7-day replay of saved data swaps the Story of the Week correctly, and the PDF is 3 to 4 pages. **Security exit** NEW**:** SR-7 is in place, and a tampered `state.json` fails closed. **Status (Nov 4):** implemented. The 7-day replay, the 3-to-4-page layout, and the tampered-state test pass in CI. The 10-CVE check against live NVD runs locally with `pytest -m live`. Until Phase 3, `state.json` stays local and git-ignored.
- **Phase 3, Automation and photos (Week 3, Nov 9 to Nov 15):** Actions workflow, Pages, archive, state commit, failure alerts. Then photos: at most 2 per edition (Story of the Week, top News of the Day) from the article's `og:image`, credited "Image: <Publisher>" with a link, skipped if missing or under 400 px wide, downscaled to 1,000 px. Done when 3 consecutive scheduled runs publish. **Security exit** NEW**:** SR-3, SR-8, and SR-9 are in place, T7 and T8 pass, and the OpenSSF Scorecard is 7.0 or higher.

7. Out of Scope

Other paid models; automatic model switching; email, Slack, or RSS delivery; accounts and authentication (static site, no user input); non-English sources; AI-generated images; search of past editions.